

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA1013	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Spandana	Reg. No.:	192525160
Date:		Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

- 1. Problem Analysis**
 - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
- 2. Project Structure Design**
 - Design and explain the folder structure of your project repository.
 - Justify the purpose of each major folder and file included in the repository.
- 3. Git Repository Initialization**
 - Create a Git repository for the project.
 - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
- 4. Git Branching Strategy**
 - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
 - Justify why the selected branching model is appropriate for the assigned project.
- 5. Version Control Workflow**
 - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
 - Explain the purpose of each Git operation performed.

6. **Commit History Analysis**
 - Present the commit history of the project.
 - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
 - Explain why Docker is suitable for the assigned project.
 - Identify the application components that require containerization.
8. **Dockerfile Development**
 - Design and explain the Dockerfile required to containerize the application.
 - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
 - Develop a Docker Compose configuration for the project.
 - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
 - Demonstrate the execution of the application using Docker containers.
 - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
 - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
 - Identify the challenges encountered during implementation.
 - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository &	Repository initialized	Repository and	Basic Git setup	Incorrect or incomplete Git

SIMATS ENGINEERING
(SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

GIT & DOCKER TECHNICAL ASSIGNMENT REPORT

Problem Statement 38
Intelligent Smart Port Operations and Cargo Management Platform

Field	Details
Student Name	Spandana
Registration Number	192525160
Course Code	CSA1013
Course Title	Software Engineering
CO Assessed	CO3 - Build full-stack applications with an emphasis on containerization, version control, and collaborative development
Assessment	CO3 Assessment Tool 1 - Git & Docker Technical Assignment
Faculty In-Charge	Latha
Weightage	20% Total Marks: 25
Duration	60 minutes

Department of Computer Science and Engineering
SIMATS Engineering (Saveetha School of Engineering)

Table of Content

1. Problem Analysis
2. Project Structure Design
3. Git Repository Initialization
4. Git Branching Strategy
5. Version Control Workflow
6. Commit History Analysis
7. Docker Environment Design
8. Dockerfile Development
9. Docker Compose Design
10. Container Deployment and Testing
11. Benefits of Git and Docker
12. Challenges and Improvements
13. References

1. Problem Analysis

1.1 Industry Context

Modern seaports handle high volumes of cargo and vessel traffic on a daily basis, which demands efficient logistics coordination and continuous, real-time operational monitoring. The convergence of Artificial Intelligence (AI), the Internet of Things (IoT), Radio-Frequency Identification (RFID), Global Positioning System (GPS), and cloud computing has enabled ports to move from manual, paper-based operations towards intelligent cargo tracking, automated scheduling, and predictive analytics, thereby improving overall port productivity and reducing operational delays.

1.2 Problem Statement

Port authorities frequently experience operational inefficiencies caused by cargo congestion, manual documentation, delayed vessel scheduling, and limited visibility of cargo movement across the port. These inefficiencies directly increase vessel turnaround time and overall logistics costs. The proposed Intelligent Smart Port Operations and Cargo Management Platform addresses this gap by monitoring cargo movement using IoT devices, optimizing berth allocation, predicting cargo handling delays using AI, automating documentation processes, and providing real-time operational dashboards - while enhancing logistics efficiency and supporting secure, transparent port operations.

1.3 Project Objectives

1. Monitor cargo movement in real time using IoT, RFID, and GPS data streams.
2. Optimize berth and vessel scheduling to minimise vessel waiting time.
3. Predict cargo handling delays using AI/ML models trained on historical operational data.
4. Automate logistics documentation, including manifests, customs paperwork, and EDI exchange.
5. Improve warehouse coordination between yard, gate, and storage operations.
6. Generate operational performance reports for port management and stakeholders.
7. Enhance cargo security through access control, tamper detection, and audit trails.
8. Increase overall port operational efficiency and reduce turnaround time and cost.

1.4 Functional Requirements

- The system shall ingest and process real-time telemetry from IoT sensors, RFID readers, and GPS/AIS vessel trackers.
- The system shall provide an AI-based berth allocation and vessel scheduling optimisation module.
- The system shall predict cargo handling delays using historical and live operational data.
- The system shall auto-generate and validate shipping/customs documentation (EDI, manifests, bill of lading).
- The system shall coordinate warehouse yard operations including container placement and retrieval.
- The system shall provide a real-time operational dashboard with drill-down analytics and alerts.
- The system shall implement role-based access control (Port Authority, Shipping Agent, Customs Officer, Warehouse Operator).
- The system shall maintain a tamper-evident audit trail of all cargo movement and document transactions.

1.5 Non-Functional Requirements

- Scalability - horizontally scalable microservices to handle peak vessel-arrival traffic.
- Availability - target uptime of 99.9% for mission-critical scheduling and tracking services.
- Low Latency - near real-time (sub-second to few-second) processing of IoT/RFID event streams.
- Security - encrypted data in transit/at rest, role-based access, and secure API gateways.
- Interoperability - standards-based EDI/REST integration with customs and shipping-line systems.
- Portability - containerized deployment that runs identically across development, staging, and port-site production environments.

1.6 Expected Outcomes

9. Reduced cargo handling time.
10. Improved logistics efficiency.
11. Better vessel scheduling.
12. Enhanced cargo visibility.
13. Lower operational costs.
14. Improved port productivity.
15. Increased supply chain reliability.
16. Better customer service.

2. Project Structure Design

The repository follows a monorepo-style, service-oriented folder structure so that each independently deployable microservice, the frontend dashboard, container definitions, documentation, and automated tests are clearly separated while still being version-controlled together for easier cross-service coordination.

2.1 Repository Folder Structure

```
smart-port-platform/
|-- .github/
|   +-- workflows/           # CI/CD pipelines (build, test, image push)
|-- backend/
|   |-- api-gateway/         # Nginx / Express gateway, routes to microservices
|   |-- cargo-tracking-service/ # IoT/RFID/GPS ingestion + tracking API
|   |-- berth-scheduling-service/ # Berth allocation & vessel scheduling logic
|   |-- ai-prediction-service/  # AI/ML delay-prediction microservice
|   |-- documentation-service/  # EDI / manifest / customs document automation
|   |-- warehouse-service/     # Yard & warehouse coordination
|   +-- shared/               # Shared libraries, DTOs, auth middleware
|-- frontend/
|   |-- dashboard-app/       # React operational dashboard (Port Authority)
|   +-- public/
|-- docker/
|   |-- Dockerfile.backend
|   |-- Dockerfile.frontend
|   +-- docker-compose.yml
|-- docs/
|   |-- architecture-diagram.png
|   |-- git-branching-diagram.png
|   +-- api-spec.yaml
|-- scripts/
|   +-- init-db.sql
|-- tests/
|   |-- unit/
|   +-- integration/
|-- .env.example
|-- .gitignore
|-- .gitattributes
|-- README.md
+-- LICENSE
```

2.2 Purpose of Major Folders and Files

Folder / File	Purpose
.github/workflows/	Holds GitHub Actions CI/CD pipeline definitions that automatically build, test, and push Docker images on every push/PR.
backend/api-gateway/	Single entry point that routes client requests to the correct microservice and enforces authentication.
backend/cargo-tracking-service/	Consumes IoT/RFID/GPS event streams and exposes REST/WebSocket APIs for real-time cargo location.
backend/berth-scheduling-service/	Implements optimisation logic for berth allocation and vessel scheduling.
backend/ai-prediction-service/	Hosts the AI/ML model that predicts cargo handling delays.
backend/documentation-service/	Automates generation and validation of manifests, EDI messages, and customs paperwork.
backend/warehouse-service/	Coordinates yard, gate, and storage operations.
backend/shared/	Common code (auth middleware, DTOs, logging) reused across microservices to avoid duplication.
frontend/dashboard-app/	React-based real-time operational dashboard used by port authorities and operators.
docker/	Centralises all Dockerfiles and the docker-compose.yml used to build and orchestrate every container.
docs/	Architecture diagrams, branching diagram, and the OpenAPI specification for the platform.
scripts/init-db.sql	Database bootstrap script executed automatically when the PostgreSQL container first starts.
tests/unit, tests/integration	Automated unit and integration tests executed locally and inside the CI pipeline.
.env.example	Template of required environment variables (without secrets) for local/container configuration.
.gitignore	Prevents build artefacts, node_modules, and secrets from being committed to the repository.
.gitattributes	Normalises line endings and marks binary files so that diffs remain clean across operating systems.
README.md	Project overview, setup instructions, and usage guide for new contributors.
LICENSE	Declares the legal terms under which the source code may be used and distributed.

3. Git Repository Initialization

3.1 Repository Initialization Steps

```
# 1. Create the project folder and move into it
mkdir smart-port-platform && cd smart-port-platform

# 2. Initialise a new Git repository
git init

# 3. Configure user identity (once per machine)
git config user.name "Spandana"
git config user.email "spandana@simatsengineering.edu"

# 4. Add essential files
touch README.md .gitignore .gitattributes LICENSE

# 5. Stage and create the initial commit
git add .
git commit -m "chore: initial commit - project scaffold"

# 6. Create the remote repository and link it
git remote add origin https://github.com/spandana/smart-port-platform.git
git branch -M main
git push -u origin main
```

3.2 Purpose of Essential Git Files

File / Folder	Purpose
.git/	Hidden directory created by 'git init' that stores the entire object database, refs, and history of the repository.
.gitignore	Lists files and folders (node_modules/, dist/, .env, *.log) that Git should never track or commit.
.gitattributes	Defines per-path attributes such as line-ending normalisation (LF/CRLF) and marks binary assets.
README.md	First point of reference describing the project purpose, architecture, and how to run it.
LICENSE	States the terms under which others may use, modify, or distribute the source code.
HEAD (inside .git)	Pointer to the currently checked-out branch/commit; moves automatically as branches are switched.

4. Git Branching Strategy

A Gitflow-inspired branching model was selected for the Smart Port platform because the project is composed of several independently developed microservices (cargo tracking, berth scheduling, AI prediction, documentation, warehouse) that must be integrated, tested, and released together in a controlled manner.

4.1 Branch Roles

Branch	Role
main	Always reflects the production-ready, deployed state of the platform. Only release/ and hotfix/ branches merge here.
develop	Integration branch where completed features from all microservice teams are merged and tested together.
feature/*	Short-lived branches (e.g. feature/cargo-tracking, feature/berth-scheduling, feature/ai-delay-predict) created from develop for a single objective.
release/*	Cut from develop when a version is feature-complete; used for final QA, staging deployment, and documentation before merging to main.
hotfix/*	Branched directly from main to urgently patch production issues (e.g. a security vulnerability), then merged back into both main and develop.

Git Branching Strategy - Smart Port Cargo Management Platform

feature -> develop -> release -> main | hotfix -> main & develop

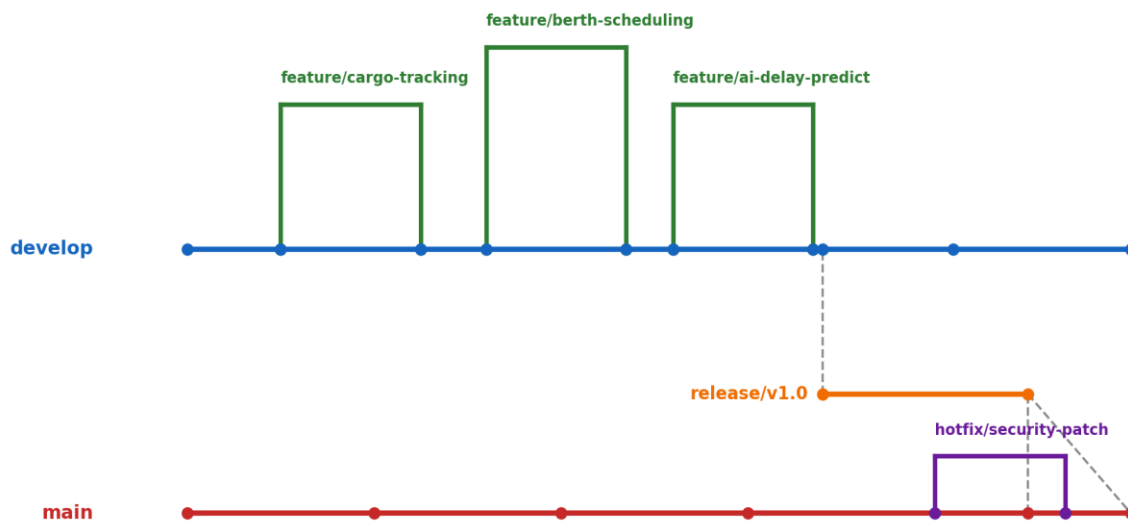


Figure 1: Git branching strategy for the Smart Port Cargo Management Platform.

4.2 Justification

- Parallel development - separate feature branches let the IoT/tracking, AI, scheduling, and documentation teams work simultaneously without blocking each other.
- Stable production - main is protected and only receives thoroughly tested code via release or hotfix branches, which is critical for a platform that coordinates live vessel and cargo operations.
- Safe integration - the develop branch acts as a continuous integration point, surfacing merge conflicts and cross-service issues early.
- Rapid incident response - the hotfix/* pattern allows a security or scheduling bug to be patched in production within minutes without pulling in unrelated, unfinished features.
- Traceable releases - release/* branches combined with version tags (v1.0, v1.1) give the port authority an auditable history of exactly what was deployed and when.

5. Version Control Workflow

The table below demonstrates the end-to-end Git workflow used while building the Berth Scheduling feature, including branch creation, meaningful commits, a merge conflict and its resolution, and synchronisation with the remote repository.

Step	Git Operation	Purpose
1	git checkout develop && git pull origin develop	Start from the latest integrated code before branching off.
2	git checkout -b feature/berth-scheduling	Create an isolated branch for the berth scheduling optimisation feature.
3	git add berth-scheduling-service/ && git commit -m "feat(berth): add greedy berth allocation algorithm"	Record a meaningful, scoped commit describing the change.
4	git push -u origin feature/berth-scheduling	Publish the feature branch to the remote so it can be reviewed and backed up.
5	git checkout develop && git pull origin develop	Re-sync with develop before merging, to pull in any teammates' changes.
6	git merge feature/berth-scheduling	Integrate the completed feature into the shared develop branch.
7	CONFLICT (content): berth-scheduling-service/config.js	Two branches modified the same configuration line - Git flags it for manual resolution.
8	Edit conflict markers, keep both DB and AI-service settings, then git add config.js	Manually resolve the conflict by combining both required changes.
9	git commit -m "merge: resolve config conflict between berth-scheduling and ai-prediction"	Finalise the merge commit once the conflict is resolved.
10	git push origin develop	Synchronise the resolved, integrated code back to the remote repository.
11	git tag -a v1.0 -m "Release 1.0 - core scheduling & tracking" && git push origin v1.0	Tag a stable point in history for release tracking.

6. Commit History Analysis

6.1 Sample Commit History

```
$ git log --oneline --graph --decorate
* 9f3a1c2 (HEAD -> main, tag: v1.0) merge: release v1.0 into main
* 7b2e4d1 (release/v1.0) chore: bump version to 1.0.0
* 4c1a8e9 docs: update README with deployment instructions
* 2d9f0a3 (develop) merge: resolve config conflict between berth-scheduling and ai-prediction
* 6a1c3f8 feat(ai): integrate delay-prediction model into gateway
* 1e7b5c0 feat(berth): add greedy berth allocation algorithm
* 8d4f2b6 feat(cargo): stream RFID/GPS events via MQTT
* 5c0a9e7 test(cargo): add unit tests for tracking service
* 3f8b1d4 fix(docs-service): correct EDI manifest field mapping
* 0a2e6c1 chore: initial commit - project scaffold
```

6.2 Commit Message Conventions

All commits follow the Conventional Commits standard, using the format `type(scope): short description`, which keeps the history searchable and machine-readable for changelog generation.

Prefix	Meaning	Example
feat	A new feature	feat(berth): add greedy berth allocation algorithm
fix	A bug fix	fix(docs-service): correct EDI manifest field mapping
docs	Documentation only changes	docs: update README with deployment instructions
test	Adding or correcting tests	test(cargo): add unit tests for tracking service
chore	Build process or tooling changes	chore: bump version to 1.0.0
merge	Merge commit combining branches	merge: resolve config conflict between services

This discipline contributes directly to project maintenance: each commit is small, scoped, and self-explanatory, which makes 'git blame', code review, and rollback of a single faulty change fast and low-risk - an important property for a platform that manages live port operations.

7. Docker Environment Design

7.1 Why Docker Is Suitable for This Project

- Microservice isolation - the platform is split into five+ independent backend services (cargo tracking, berth scheduling, AI prediction, documentation, warehouse); Docker lets each run in its own isolated, versioned container.
- Environment parity - IoT simulators, the AI model runtime, and Node.js services often need different library versions; containers guarantee the exact same environment on a developer laptop, the CI server, and the port-site production servers.
- Rapid, repeatable deployment - a new port terminal can be brought online by running 'docker compose up' instead of manually installing dependencies on physical servers.
- Horizontal scalability - during peak vessel-arrival periods, the berth-scheduling and cargo-tracking containers can be scaled out independently of lower-traffic services.
- Simplified onboarding - new student/developer team members can get the full multi-service stack running with a single command instead of a long manual setup guide.

7.2 Components Requiring Containerization

Component	Container Image (base)
Frontend Dashboard	node:20-alpine (build) -> nginx:1.27-alpine (serve)
API Gateway	node:20-alpine
Cargo Tracking Service	node:20-alpine
Berth Scheduling Service	node:20-alpine
AI Prediction Service	python:3.11-slim
Documentation Service	node:20-alpine
Warehouse Coordination Service	node:20-alpine
MQTT Broker (IoT ingestion)	eclipse-mosquitto:2
Relational Database	postgres:16-alpine
Document Store	mongo:7
Cache / Realtime Store	redis:7-alpine

8. Dockerfile Development

A multi-stage Dockerfile is used for each Node.js microservice (illustrated below for the Cargo Tracking Service) to keep the final production image small and secure by discarding build-time dependencies.

8.1 Sample Dockerfile - backend/cargo-tracking-service/Dockerfile

```
# ---- Stage 1: Build ----
FROM node:20-alpine AS build
WORKDIR /app
```

```

COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
RUN npm run build

# --- Stage 2: Production runtime ---
FROM node:20-alpine
WORKDIR /app
ENV NODE_ENV=production
RUN addgroup -S appgrp && adduser -S appuser -G appgrp
COPY --from=build /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
COPY --from=build /app/package*.json ./
USER appuser
EXPOSE 4001
HEALTHCHECK --interval=30s --timeout=5s CMD wget -qO- http://localhost:4001/health || exit 1
CMD ["node", "dist/index.js"]

```

8.2 Instruction-by-Instruction Explanation

Instruction	Purpose
FROM node:20-alpine AS build	Starts the build stage from a lightweight Node.js 20 Alpine base image, named 'build' for later reference.
WORKDIR /app	Sets /app as the working directory inside the container for all subsequent instructions.
COPY package*.json ./	Copies only the dependency manifests first so Docker can cache the npm-install layer.
RUN npm ci --omit=dev	Performs a clean, reproducible install of production dependencies only.
COPY . .	Copies the remaining application source code into the image.
RUN npm run build	Transpiles/bundles the TypeScript source into the dist/ output folder.
FROM node:20-alpine (2nd stage)	Starts a fresh, minimal runtime image, discarding build tools and dev dependencies from stage 1.
ENV NODE_ENV=production	Configures the Node.js runtime to run in optimised production mode.
RUN addgroup/adduser	Creates a dedicated non-root user so the container does not run as root (security hardening).
COPY --from=build ...	Copies only the compiled output and node_modules from the build stage, keeping the final image small.
USER appuser	Switches the running process to the non-root user created earlier.
EXPOSE 4001	Documents the port the cargo-tracking API listens on inside the container network.
HEALTHCHECK	Lets Docker/Compose automatically detect and report an unhealthy container via its /health endpoint.
CMD ["node", "dist/index.js"]	Defines the default command executed when the container starts.

9. Docker Compose Design

Docker Compose orchestrates all eleven containers of the Smart Port platform as a single multi-service application, wiring up an isolated network, persistent volumes, and environment configuration so the entire stack starts with one command.

9.1 docker-compose.yml

```
version: "3.9"

services:
  frontend:
    build: ./frontend/dashboard-app
    ports: ["3000:80"]
    depends_on: [api-gateway]
    networks: [port-net]

  api-gateway:
    build:
      context: ./backend/api-gateway
    env_file: .env
    ports: ["4000:4000"]
    depends_on: [cargo-tracking, berth-scheduling, ai-prediction, documentation]
    networks: [port-net]

  cargo-tracking:
    build: ./backend/cargo-tracking-service
    env_file: .env
    depends_on: [postgres, mongodb, mosquito]
    networks: [port-net]

  berth-scheduling:
    build: ./backend/berth-scheduling-service
    env_file: .env
    depends_on: [postgres, redis]
    networks: [port-net]

  ai-prediction:
    build: ./backend/ai-prediction-service
    env_file: .env
    depends_on: [mongodb]
    networks: [port-net]

  documentation:
    build: ./backend/documentation-service
    env_file: .env
    depends_on: [postgres]
    networks: [port-net]

  mosquito:
    image: eclipse-mosquitto:2
    ports: ["1883:1883"]
    volumes: ["/docker/mosquitto.conf:/mosquitto/config/mosquitto.conf"]
    networks: [port-net]
```



```

postgres:
  image: postgres:16-alpine
  environment:
    POSTGRES_DB: smart_port
    POSTGRES_USER: ${DB_USER}
    POSTGRES_PASSWORD: ${DB_PASSWORD}
  volumes: ["pgdata:/var/lib/postgresql/data", "./scripts/init-db.sql:/docker-entrypoint-initdb.d/init.sql"]
  networks: [port-net]

mongodb:
  image: mongo:7
  volumes: ["mongodata:/data/db"]
  networks: [port-net]

redis:
  image: redis:7-alpine
  volumes: ["redisdata:/data"]
  networks: [port-net]

networks:
  port-net:
    driver: bridge

volumes:
  pgdata:
  mongodata:
  redisdata:

```

9.2 Explanation of Key Elements

Element	Explanation
services	Each block defines one container: the frontend, four backend microservices, the MQTT broker, and three data stores.
build / image	Application services are built from local Dockerfiles; third-party components (Postgres, MongoDB, Redis, Mosquitto) use official published images.
networks: port-net	A private bridge network so services can reach each other by service name (e.g. <code>http://cargo-tracking:4001</code>) without exposing internal ports to the host.
ports	Only the frontend (3000), API gateway (4000), and MQTT broker (1883) are published to the host; internal services stay unreachable from outside for security.
depends_on	Declares startup ordering, e.g. <code>api-gateway</code> waits for the four backend microservices; <code>cargo-tracking</code> waits for Postgres, MongoDB, and Mosquitto.
env_file: .env	Injects environment-specific configuration (DB credentials, API keys, service URLs) without hard-coding secrets into the images.
volumes (pgdata, mongodata, redisdata)	Named volumes persist database data across container restarts and rebuilds.
scripts/init-db.sql mount	Automatically initialises the PostgreSQL schema the first time the postgres container starts.

10. Container Deployment and Testing

10.1 Building and Running the Stack

```
$ docker compose build
$ docker compose up -d
[+] Running 11/11
- Network smart-port-platform_port-net    Created
- Container smart-port-postgres           Started
- Container smart-port-mongodb            Started
- Container smart-port-redis              Started
- Container smart-port-mosquitto          Started
- Container smart-port-cargo-tracking      Started
- Container smart-port-berth-scheduling   Started
- Container smart-port-ai-prediction       Started
- Container smart-port-documentation       Started
- Container smart-port-api-gateway         Started
- Container smart-port-frontend           Started
```

10.2 Verifying Container Status

```
$ docker compose ps
NAME                                STATUS                                PORTS
smart-port-frontend                 Up 2 minutes                         0.0.0.0:3000->80/tcp
smart-port-api-gateway               Up 2 minutes (healthy)               0.0.0.0:4000->4000/tcp
smart-port-cargo-tracking            Up 2 minutes (healthy)               4001/tcp
smart-port-berth-scheduling           Up 2 minutes (healthy)               4002/tcp
smart-port-ai-prediction              Up 2 minutes (healthy)               4003/tcp
smart-port-documentation              Up 2 minutes (healthy)               4004/tcp
smart-port-postgres                  Up 2 minutes                         5432/tcp
smart-port-mongodb                   Up 2 minutes                         27017/tcp
smart-port-redis                     Up 2 minutes                         6379/tcp
smart-port-mosquitto                 Up 2 minutes                         0.0.0.0:1883->1883/tcp
```

10.3 Functional Testing Procedure

- Health checks - each microservice exposes a /health endpoint; docker compose ps confirms all report (healthy).
- API smoke test - curl http://localhost:4000/api/cargo/track/CNT12345 is sent through the gateway to the cargo-tracking service and returns live location data.
- IoT ingestion test - a simulated RFID/GPS event is published to the mosquitto broker; the dashboard updates the cargo position within two seconds.
- AI prediction test - a sample vessel/berth payload is posted to the ai-prediction service; the returned predicted-delay value is validated against the expected test-case range.
- Integration tests - the tests/integration suite runs inside the CI pipeline against the fully composed stack (docker compose up in GitHub Actions) before every merge to develop.

```
$ curl -s http://localhost:4000/api/cargo/track/CNT12345
{"containerId":"CNT12345","status":"IN_TRANSIT","berth":"B-04","lastSeen":"2026-08-17T09:12:44Z","predictedDelayMinutes":8}
```

```
$ docker compose logs cargo-tracking --tail 5
cargo-tracking | INFO  MQTT event received: CNT12345 @ GPS(13.0827,80.2707)
cargo-tracking | INFO  Position persisted to MongoDB in 12ms
cargo-tracking | INFO  WebSocket broadcast to 3 dashboard client(s)
```

10.4 Teardown

```
$ docker compose down          # stop and remove containers + network
$ docker compose down -v      # also remove named volumes (fresh reset)
```

11. Benefits of Git and Docker

Benefit	How It Applies to the Smart Port Platform
Collaboration	Feature branches let the IoT/tracking, AI, scheduling, and documentation sub-teams work in parallel, with pull requests providing structured code review before merging into develop.
Version Management	Every change to scheduling logic, AI models, or EDI formats is tracked, tagged (v1.0, v1.1...), and revertible - critical when a port authority needs to audit exactly what changed between releases.
Portability	Docker images guarantee the AI-prediction service and all Node.js microservices run identically on a developer laptop, the CI runner, and production servers at different port terminals.
Deployment Speed	A new port site can be provisioned by pulling versioned images and running 'docker compose up', replacing a multi-day manual server setup with a process that takes minutes.
Scalability	Individual containers (e.g. cargo-tracking, berth-scheduling) can be scaled independently during peak vessel-arrival windows without redeploying the entire platform.
Maintainability	Small, well-scoped commits and isolated containers make it straightforward to trace, fix, and redeploy a single faulty service without affecting the rest of the platform.
Disaster Recovery	Git history plus versioned Docker images allow the platform to be rolled back to any previously known-good release quickly in the event of a production incident.

12. Challenges and Improvements

12.1 Challenges Encountered

- Cross-service merge conflicts - shared configuration files (e.g. service discovery settings) were occasionally edited by two feature branches simultaneously, requiring careful manual conflict resolution.
- Container orchestration complexity - coordinating start-up order and health dependencies across eleven containers (databases, broker, five services, frontend) needed careful use of depends_on and health checks.
- High-volume IoT data - simulating and processing continuous RFID/GPS event streams through the MQTT broker created noticeable load during local integration testing.
- AI model versioning - keeping the AI-prediction container's model artefact in sync with the training pipeline required a disciplined tagging convention separate from application code tags.
- Secrets management - ensuring database and API credentials were never committed to Git while still being easily available to every container required strict .env / .gitignore discipline.

12.2 Suggested Improvements and Future Enhancements

- Adopt Kubernetes for production-grade orchestration, auto-scaling, and self-healing beyond what Docker Compose offers for a single host.
- Extend the GitHub Actions pipeline with automated linting, unit/integration tests, and vulnerability scanning of every Docker image before it is pushed to a registry.
- Introduce a secrets manager (e.g. HashiCorp Vault or Docker Secrets) instead of plain .env files for production deployments.
- Add a service mesh (e.g. Istio) for fine-grained traffic control, observability, and mutual TLS between the five backend microservices.

- Implement blue-green or canary deployment strategies to release new berth-scheduling or AI-prediction versions with zero downtime to live port operations.
- Expand automated integration tests to cover full end-to-end IoT-to-dashboard scenarios inside the CI pipeline.

References

- Chacon, S. and Straub, B., Pro Git, 2nd Edition, Apress, 2014.
- Docker Inc., Docker Documentation - <https://docs.docker.com/>
- Docker Compose Specification - <https://docs.docker.com/compose/compose-file/>
- GitHub, Understanding the Gitflow Workflow - <https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow>
- Conventional Commits Specification - <https://www.conventionalcommits.org/>
- Mosquitto MQTT Broker Documentation - <https://mosquitto.org/documentation/>
- PostgreSQL, MongoDB, and Redis Official Docker Images - <https://hub.docker.com/>